

Spring boot - 6

6. REST 서비스 생성하기

[Spring boot - 5](#)

6.0 용어 정리

REST API란?

REpresentational State Transfer의 약자로, Web 기반의 자원에 대해 HTTP 프로토콜을 활용하여 데이터를 요청하고 응답받는 아키텍처 스타일이다. 이 스타일을 따르는 API를 RESTful API라고 한다.

REST API의 핵심 개념

1. 자원 (Resource)

URI(Uniform Resource Identifier)로 식별하는 데이터를 의미한다.

예시 : `/users`, `/posts/1`

2. 표현 (Representation)

자원의 상태를 JSON, XML 등으로 표현하여 전달한다.

3. 상태 전이 (State Transfer)

클라이언트는 서버의 자원 상태를 HTTP 요청으로 변경하고, 응답을 통해 새로운 상태를 얻는다.

4. 무상태성 (Stateless)

요청 간 서버가 클라이언트의 상태를 저장하지 않는다. 즉, 모든 요청이 독립적이다.

REST API의 구성 요소

1. HTTP Method

자원에 대한 행위를 명시한다.

- GET : 조회
- POST : 생성
- PUT : 전체 수정
- PATCH : 부분 수정
- DELETE : 삭제

2. URI

어떤 자원을 대상으로 하는지 식별한다.

예시: `/users`, `/users/123`

3. HTTP 상태 코드

상태 코드	의미
200	OK - 성공
201	Created - 자원 생성 완료
204	No Content - 성공했으나 반환할 데이터 없음
400	Bad Request - 잘못된 요청
401	Unauthorized - 인증 실패
404	Not Found - 자원을 찾을 수 없음
500	Internal Server Error - 서버 오류

REST API의 실제 활용

보통 프론트엔드와 백엔드로 분리된 구조에서 사용된다.

1. 프론트엔드 (Vue.js, React, Angular 등)

사용자가 `/users/123`에 접근하면 JavaScript가 API를 호출한다.

```
GET /api/users/123
```

2. 백엔드

서버는 사용자 정보를 데이터베이스에서 조회하여 JSON으로 응답한다.

```
{
  "id": 123,
  "name": "홍길동",
  "email": "hong@example.com"
}
```

SPA와 MPA

SPA (Single Page Application)

- 페이지 1개로 모든 화면을 구성한다.
- 첫 페이지 로딩 후, JavaScript가 동적으로 View만 교체한다.
- 페이지 이동이 없으며 Router가 동작한다.

MPA (Multi Page Application)

- 여러 개의 HTML 페이지로 구성된다.
- 사용자 요청마다 전체 페이지를 새로 로딩한다.
- 실제로 다른 URL로 이동하며 새로고침이 발생한다.

SPA vs MPA 비교

구분	SPA	MPA
초기 로딩 속도	느림 (JS 파일이 큼)	빠름
페이지 전환 속도	빠름 (View만 교체)	느림 (전체 페이지 재요청)
서버 요청	API 요청만	페이지 리소스 재요청
개발 방식	클라이언트 중심 (Vue, React)	서버 중심 (JSP, Thymeleaf)
상태 관리	클라이언트가 직접 관리	페이지마다 초기화
보안	JS 노출로 공격면이 넓음	서버 처리 중심

프론트엔드의 Router

브라우저 주소에 따라 보여줄 페이지를 결정하는 JavaScript 라이브러리다.

- URL이 바뀌면 전체 페이지를 새로고침하지 않고 화면 구성만 바꿔준다.
- 사용자에게는 진짜 페이지가 이동하는 것처럼 보이지만, 실제로는 SPA의 한 페이지 안에서 View가 바뀐 것뿐이다.

```
import Home from './Home.vue'
import About from './About.vue'

export default [
  { path: '/', component: Home },
  { path: '/about', component: About }
]
```

사용자가 /about URL에 접근하면 라우터가 About.vue 를 화면에 보여준다. 실제로 HTML을 새로 요청하지는 않는다.

본 장에서는 스프링을 사용하여 애플리케이션에 REST API를 제공하는 방법을 다룬다. 스프링 MVC 컨트롤러를 사용해서 REST 엔드포인트를 생성하며, 스프링 데이터 리포지토리의 REST 엔드포인트

도 외부에서 사용할 수 있게 자동으로 노출시킨다.

6.1 REST 컨트롤러 작성하기

클라이언트 코드는 HTTP 요청을 통해 REST API와 통신한다. 스프링 MVC는 다양한 타입의 HTTP 요청에 사용되는 어노테이션들을 제공한다.

HTTP 메소드별 어노테이션

어노테이션	HTTP 메소드	용도
@GetMapping	GET	리소스 데이터 읽기
@PostMapping	POST	리소스 생성
@PutMapping	PUT	리소스 전체 변경
@PatchMapping	PATCH	리소스 부분 변경
@DeleteMapping	DELETE	리소스 삭제
@RequestMapping	다목적	HTTP 메소드를 속성에 지정

REST API 컨트롤러 예제

Request DTO

```
@Data
@AllArgsConstructor
public class CreateMemberRequest {

    @NotBlank
    @NotNull(message = "로그인 아이디가 Null이면 안됩니다")
    private String mbLoginId;

    @NotNull(message = "비밀번호가 Null이면 안됩니다")
    private String mbPw;

    @NotNull(message = "기관명이 Null이면 안됩니다")
    private String organization;

    @NotNull(message = "부서명이 Null이면 안됩니다")
    private String deptName;

    @NotNull(message = "팀명이 Null이면 안됩니다")
    private String teamName;
}
```

Service Layer

```
@Service
public class MemberCommandServiceImpl implements MemberCommandService {

    private final MemberRepo mbRepo;
    private final PasswordEncoder passwordEncoder;

    public MemberCommandServiceImpl(MemberRepo mbRepo, PasswordEncoder
passwordEncoder) {
        this.mbRepo = mbRepo;
        this.passwordEncoder = passwordEncoder;
    }

    @Override
    public CreateMemberResponse createMember(CreateMemberRequest dto) {
        Members m = Members.createMembers(
            MemberLoginId.of(dto.getMbLoginId()),
            MemberPassword.of(passwordEncoder.encode(dto.getMbPw())),
            MemberOrganization.of(dto.getOrganization()),
            MemberDepartmentName.of(dto.getDeptName()),
            MemberTeamName.of(dto.getTeamName()),
            Members.MemberRole.ROLE_GENERAL_USER,
            1
        );

        MemberEntity entity = MemberConverter.toEntity(m);
        MemberEntity result = mbRepo.save(entity);

        if(result == null) {
            return new CreateMemberResponse(false, -1L);
        }
        return new CreateMemberResponse(true, result.getMbId());
    }
}
```

REST Controller

```
@RestController
@RequestMapping("/api/v1/signup")
public class RestMemberSignupController {

    private final MemberCommandService memberCommandService;

    public RestMemberSignupController(MemberCommandService memberCommandService)
{
        this.memberCommandService = memberCommandService;
    }

    @PostMapping
    public ResponseEntity<CreateMemberResponse> signup(@Valid @RequestBody
```

```
CreateMemberRequest request) {  
    CreateMemberResponse response =  
    memberCommandService.createMember(request);  
    return ResponseEntity.ok(response);  
}  
}
```

주요 어노테이션 설명

@RestController

- REST API 전용 컨트롤러임을 명시한다.
- 내부적으로 @Controller + @ResponseBody 가 합쳐진 구조다.
- 반환 값을 View로 해석하지 않는다.
- 반환 값은 HTTP Response Body(JSON, XML 등)에 바로 넣어진다.

@RequestMapping

요청 URL과 메소드 매핑을 정의한다.

- 클래스 레벨 또는 메소드 레벨에 붙인다.
- path / value : 매핑할 경로
- method : HTTP Method
- produces : 응답의 Content-Type 명시
- consumes : 요청의 Content-Type 제한

```
@RequestMapping(  
    path = "/api/v1/signup",  
    produces = {"application/json", "text/xml"}  
)
```

@CrossOrigin

CORS(Cross-Origin Resource Sharing)

브라우저가 서로 다른 출처(Origin) 간에 자원을 요청할 때 적용되는 보안 메커니즘이다.

Origin(출처) = 프로토콜 + 도메인 + 포트

이 중 하나라도 다르면 다른 출처로 취급되어 CORS 규칙이 적용된다.

브라우저는 보안상 기본적으로 **Same-Origin Policy(동일 출처 정책)**을 따른다. 즉, 다른 출처의 자원을 마음대로 요청하지 못하도록 막는다.

```
프론트엔드 : http://localhost:3000  
백엔드 API : http://localhost:8080
```

이런 식으로 서로 다른 출처를 사용하는 경우가 많기 때문에, 백엔드 서버에서 CORS 허용 설정을 해줘야 정상적으로 통신할 수 있다.

@CrossOrigin 은 CORS 허용 설정을 할 수 있다.

- `origins` : 허용할 도메인
- `methods` : 허용할 HTTP 메소드
- `allowedHeaders` : 허용할 헤더
- `maxAge` : preflight 요청 캐시 시간

```
@CrossOrigin(origins = "http://localhost:3000")
```

REST API 동작 흐름

클라이언트에서 `/api/v1/signup` 에 POST 요청을 보내면 `signup` 메소드가 실행된다.

1. HTTP 요청의 JSON 데이터가 `CreateMemberRequest` 에 자동으로 바인딩된다.
2. `@RequestBody` 를 통해 요청 본문을 Java 객체로 변환한다.
3. `@Valid` 를 통해 유효성 검증이 수행된다.
4. `createMember()` 메소드가 동작하여 회원을 생성한다.
5. 결과에 맞게 `CreateMemberResponse` 로 응답을 보낸다.

핵심 어노테이션 상세

@RequestBody

- HTTP 요청의 body(JSON, XML 등)를 Java 객체로 변환하여 바인딩한 뒤 파라미터에 주입한다.
- 변환은 `HttpMessageConverter` (보통 Jackson JSON Converter)가 담당한다.
- 즉, JSON → Java 객체로 자동 역직렬화(Deserialize)한다.

@Valid

- 요청으로 들어온 객체에 대해 Bean Validation(JSR-380) 검증을 실행한다.
- `CreateMemberRequest` 클래스의 `@NotNull`, `@Size`, `@Email` 같은 어노테이션을 검사한다.
- 검증 실패 시 `MethodArgumentNotValidException` 발생 → 400 Bad Request로 처리된다.

ResponseEntity<T>

- HTTP 응답 상태 코드, 헤더, 바디를 직접 제어할 수 있는 응답 객체다.
- `ResponseEntity.ok(body)` → 200 OK + body 반환

Spring Web MVC 요청 처리 흐름

1. 클라이언트가 `/api/v1/signup` 으로 POST 요청 전송
2. `DispatcherServlet` 이 요청을 받음 (스프링 MVC의 핵심 프론트 컨트롤러)
3. `HandlerMapping` → 어떤 Controller 메소드가 처리할지 결정 (`@PostMapping` 기준)
4. `HandlerAdapter` → 메소드 호출 준비 (파라미터에 `@RequestBody`, `@Valid` 등 확인)
5. `HttpMessageConverter` 가 JSON → Java 객체 변환 (`MappingJackson2HttpMessageConverter` 작동)
6. Bean Validation 실행 (`@Valid`)
7. Controller 메소드 실행
8. 반환값(`ResponseEntity`)을 `HttpMessageConverter` 가 Java → JSON 변환하여 응답

@ModelAttribute VS @RequestBody 비교

구분	@ModelAttribute	@RequestBody
데이터 추출 위치	쿼리 파라미터, 폼 데이터, 경로 변수	HTTP Body (JSON/XML)
데이터 형식	key-value (폼), query string	JSON, XML, Raw body
사용 방식	웹 폼 기반 요청에 적합	REST API의 JSON 기반 요청에 적합
변환 방식	<code>WebDataBinder</code> (필드 단위 바인딩)	<code>HttpMessageConverter</code> (전체 body 변환)
예시 요청	<code>/signup?name=홍길동&age=20</code>	<code>{ "name": "홍길동", "age": 20 }</code>
Validation	<code>@Valid</code> 적용 가능	<code>@Valid</code> 적용 가능

@ModelAttribute 예시

```
POST /signup
Content-Type: application/x-www-form-urlencoded

name=홍길동&age=20
```

```
@PostMapping("/signup")
public void signup(@ModelAttribute CreateMemberRequest req) {
    // ...
}
```

@RequestBody 예시

```
POST /signup
Content-Type: application/json

{
  "name": "홍길동",
  "age": 20
}
```



```
@PostMapping("/signup")
public void signup(@RequestBody CreateMemberRequest req) {
    // ...
}
```

경로 변수를 사용한 조회 API

특정 ID로 데이터를 가져오는 엔드포인트를 제공하려면 메소드의 경로에 플레이스홀더 변수를 지정하고, 해당 변수를 통해 ID를 인자로 받는 메소드를 추가하면 된다.

```
@GetMapping("/{id}")
public ResponseEntity<MemberResponse> getMemberById(@PathVariable Long id) {
    MemberResponse response = new MemberResponse(id, "홍길동");
    return ResponseEntity.ok(response);
}
```

@PathVariable

- URL 경로에 포함된 값을 파라미터로 받아오는 어노테이션이다.
- REST API에서 자주 쓰는 path parameter 매핑할 때 사용한다.
- URL 경로 변수 이름과 메소드 파라미터 이름이 같으면 생략 가능하고, 다르면 명시해야 한다.

```
@GetMapping("/posts/{postId}")
public String getPost(@PathVariable("postId") Long id) {
    // ...
}
```

ResponseEntity 상세

Spring에서 HTTP 응답 전체를 다루기 위해 제공하는 클래스다. 단순히 데이터만 리턴하는 것이 아니라 상태 코드, 헤더, 바디 모두를 제어할 수 있다.

- 패키지: org.springframework.http.ResponseEntity
- 제네릭 타입 <T> 는 응답 Body에 들어갈 데이터 타입이다.
- 보통 REST API에서 세밀하게 응답을 제어할 때 사용한다.

```
return ResponseEntity
    .status(HttpStatus.OK) // 상태 코드
    .header("Custom-Header", "myValue") // 응답 헤더
    .body("hello"); // 응답 바디 (JSON으로 자동 직렬화)
```

HTTP 요청 구조

```
GET /users/123 HTTP/1.1      ← 요청 라인
Host: example.com           ← 헤더
User-Agent: Mozilla/5.0      ← 헤더
Accept: application/json     ← 헤더
                              ← 빈 줄

(GET 요청은 주로 바디가 없음)
```

1. 요청 라인

HTTP 메소드 + 요청 URI + 프로토콜 버전

2. 요청 헤더

요청에 대한 부가 정보 (Host, User-Agent, Content-Type, Accept, Authorization 등)

3. 요청 바디

실제 서버에 전송할 데이터 (JSON, XML 등)

HTTP 응답 구조

```
HTTP/1.1 200 OK              ← 상태 라인
Content-Type: application/json ← 헤더
Content-Length: 48            ← 헤더
X-Custom-Header: MyValue     ← 헤더
                              ← 빈 줄
{                             ← 바디
  "id": 123,
  "name": "홍길동"
}
```

1. 상태 라인

프로토콜 버전 + 상태 코드 + 상태 메시지

2. 응답 헤더

응답에 대한 설명 (형식, 캐싱, 인증, 쿠키 등)

3. 응답 바디

실제 데이터

6.1.2 서버에 데이터 전송하기

```

@RestController
@RequestMapping("/api/v1/signup")
public class RestMemberSignupController {

    private final MemberCommandService memberCommandService;

    public RestMemberSignupController(MemberCommandService memberCommandService)
    {
        this.memberCommandService = memberCommandService;
    }

    @PostMapping
    public ResponseEntity<CreateMemberResponse> signup(@Valid @RequestBody
    CreateMemberRequest request) {
        CreateMemberResponse response =
        memberCommandService.createMember(request);
        return ResponseEntity.ok(response);
    }
}

```

HTTP POST 요청을 처리하므로 @PostMapping 어노테이션을 지정하였다. path 경로를 지정하지 않았으므로 클래스 레벨의 "/api/v1/signup" 에 대한 요청을 처리한다.

여기서 중요한 부분은 @RequestBody 어노테이션이다. 이것이 지정되지 않으면 스프링 MVC는 요청 파라미터(쿼리 파라미터나 폼 파라미터)가 곧바로 CreateMemberRequest 객체와 바인딩되는 것으로 간주하기 때문이다. @RequestBody 가 있어야 HTTP 요청 본문의 JSON을 객체로 변환한다.

6.1.3 서버의 데이터 변경하기

데이터를 변경하기 위한 HTTP 메소드로는 PUT과 PATCH가 있다. 각각의 차이점을 먼저 알아보자.

PUT은 데이터를 변경하는 데 사용되지만, 실제로는 GET과 반대의 의미를 갖는다. GET 요청은 서버로부터 클라이언트로 데이터를 전송하는 반면, PUT 요청은 클라이언트로부터 서버로 데이터를 전송한다.

PUT은 데이터 전체를 교체하는 것이며, PATCH는 데이터의 일부분을 변경하는 것이다.

PUT vs PATCH

PUT

- 리소스의 전체를 새 값으로 교체한다.
- 요청 바디에서 명시되지 않은 필드는 초기화되거나 삭제될 수 있다.
- 클라이언트가 리소스의 전체 상태를 알고 있어야 한다.

PATCH

- 리소스의 일부만 수정한다.
- 요청 바디에 포함된 필드만 업데이트하고, 나머지는 그대로 유지한다.

- 클라이언트가 전체를 알 필요가 없다.

예제 코드

```
public class MemberUpdateRequest {
    private String name;
    private Integer age;

    // getters/setters 생략
}

@PutMapping("/members/{id}")
public ResponseEntity<MemberUpdateRequest> updateMember(
    @PathVariable Long id,
    @RequestBody MemberUpdateRequest request
) {
    // id를 가진 Member를 request 전체 데이터로 교체
    // 실제로는 DB에서 덮어쓰기 수행

    return ResponseEntity.ok(request);
}

@PatchMapping("/members/{id}")
public ResponseEntity<MemberUpdateRequest> patchMember(
    @PathVariable Long id,
    @RequestBody MemberUpdateRequest request
) {
    // 기존 Member의 특정 필드만 변경
    // (name만 요청에 포함되었으면 name만 수정)

    return ResponseEntity.ok(request);
}
```

비교표

메소드	의미	특징
PUT	전체 교체	모든 필드를 새 값으로 교체. 빠진 필드는 null이 될 수 있음
PATCH	부분 수정	변경된 필드만 업데이트. 나머지는 유지

6.1.4 서버에서 데이터 삭제하기

```
@RestController
@RequestMapping("/members")
public class MemberController {
```

```
@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteMember(@PathVariable Long id) {
    // id로 회원을 삭제
    // memberService.delete(id);

    return ResponseEntity.noContent().build(); // 204 No Content 반환
}
}
```

데이터를 삭제할 때 클라이언트에서는 HTTP DELETE 요청으로 삭제를 요청하면 된다. DELETE 요청을 처리하는 메소드에 스프링 MVC의 @DeleteMapping 을 지정한다.

이 메소드는 데이터를 삭제하는 것이므로 클라이언트에게 데이터를 반환할 필요가 없다. 따라서 대개의 경우 DELETE 요청의 응답은 본문 데이터를 갖지 않으며, 반환 데이터가 없다는 것을 클라이언트가 알 수 있게 HTTP 상태 코드 204 No Content 를 사용한다.
